

SPAERS

Smart Panic Alert & Emergency Response System

Figures and Diagrams

Companion to the project thesis. Screenshots are not included — only architectural, structural, and behavioural diagrams.



Figure 3.1: Waterfall SDLC Phases

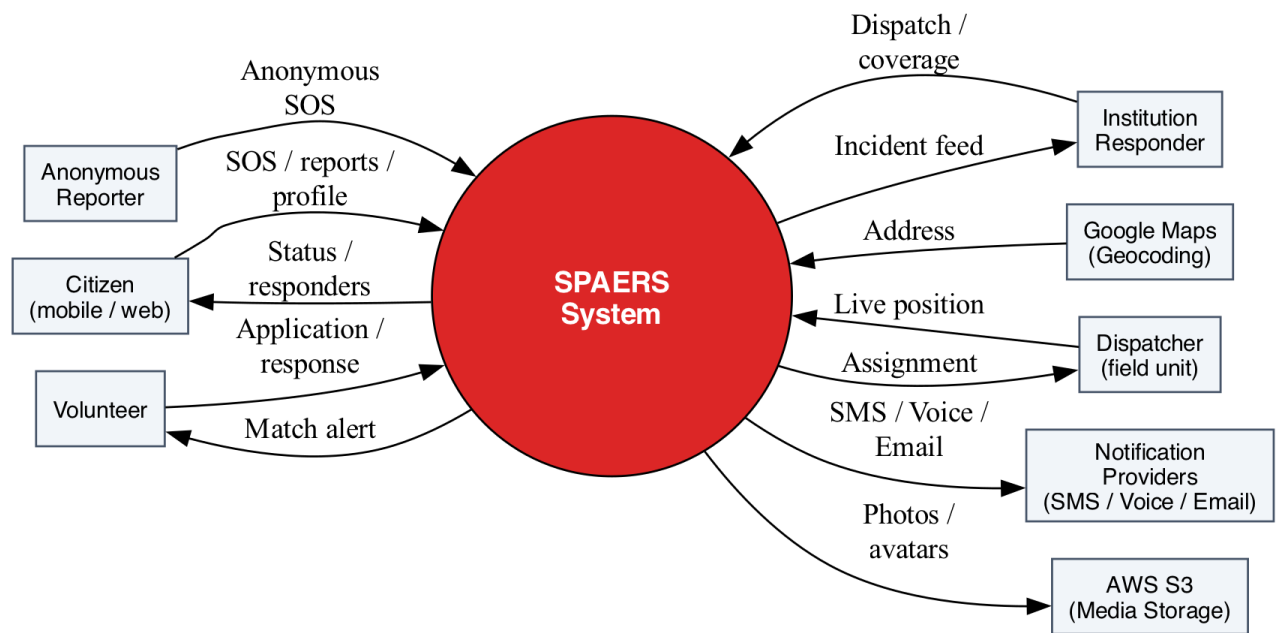


Figure 3.2: Context Diagram (Level 0 DFD)

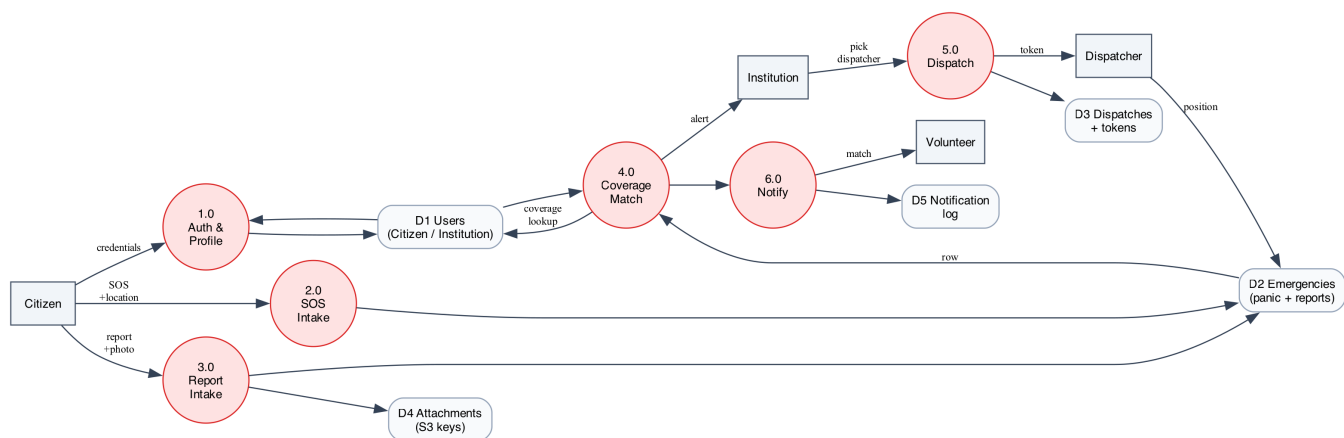


Figure 3.3: Level 1 Data Flow Diagram

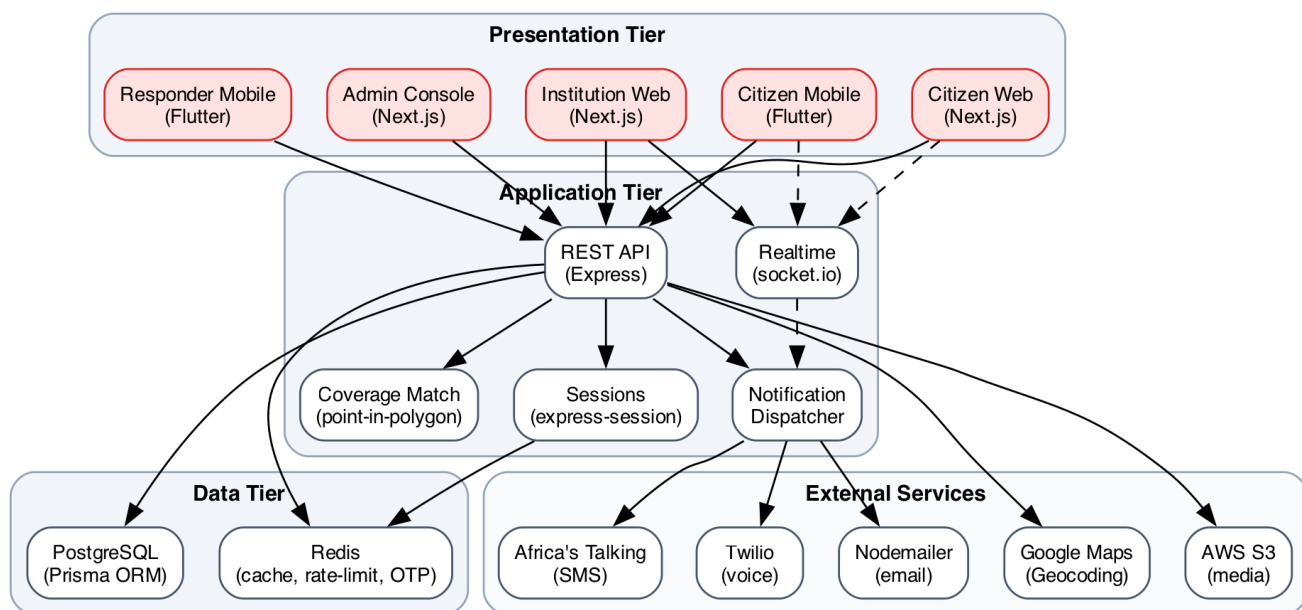


Figure 3.4: Three-Tier System Architecture

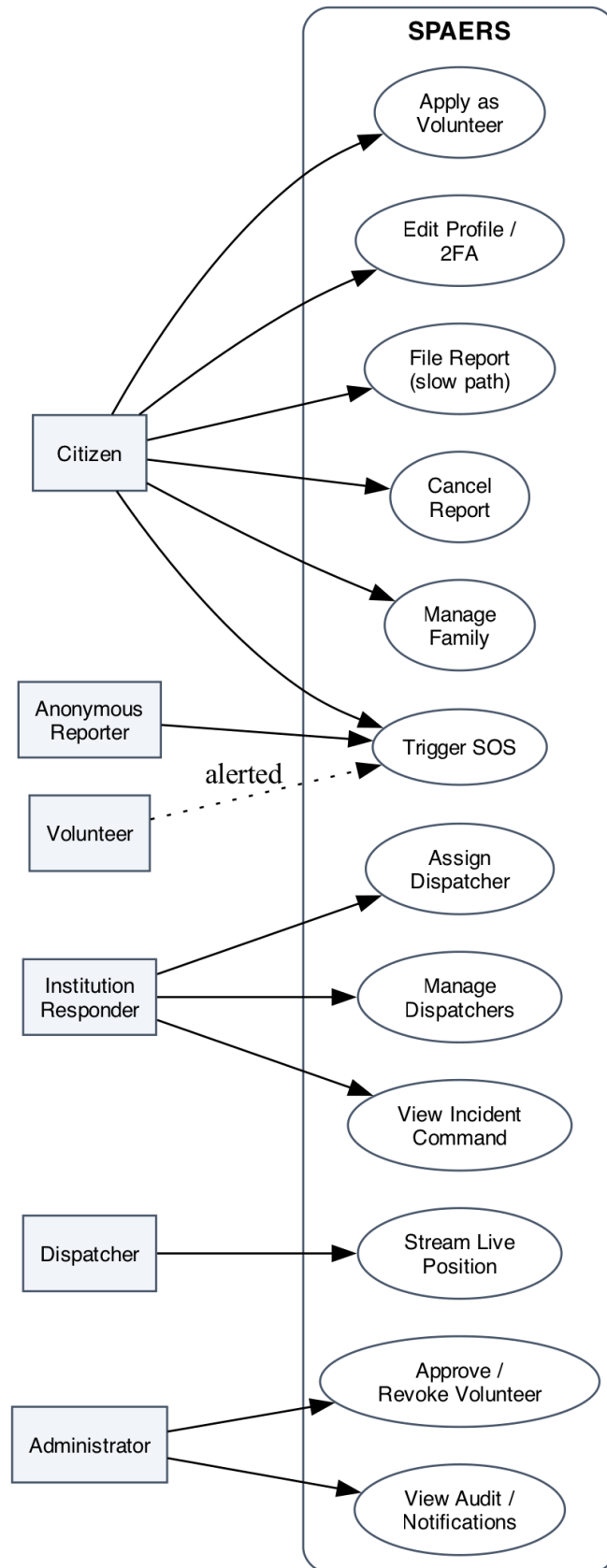


Figure 3.5: System Use Case Diagram

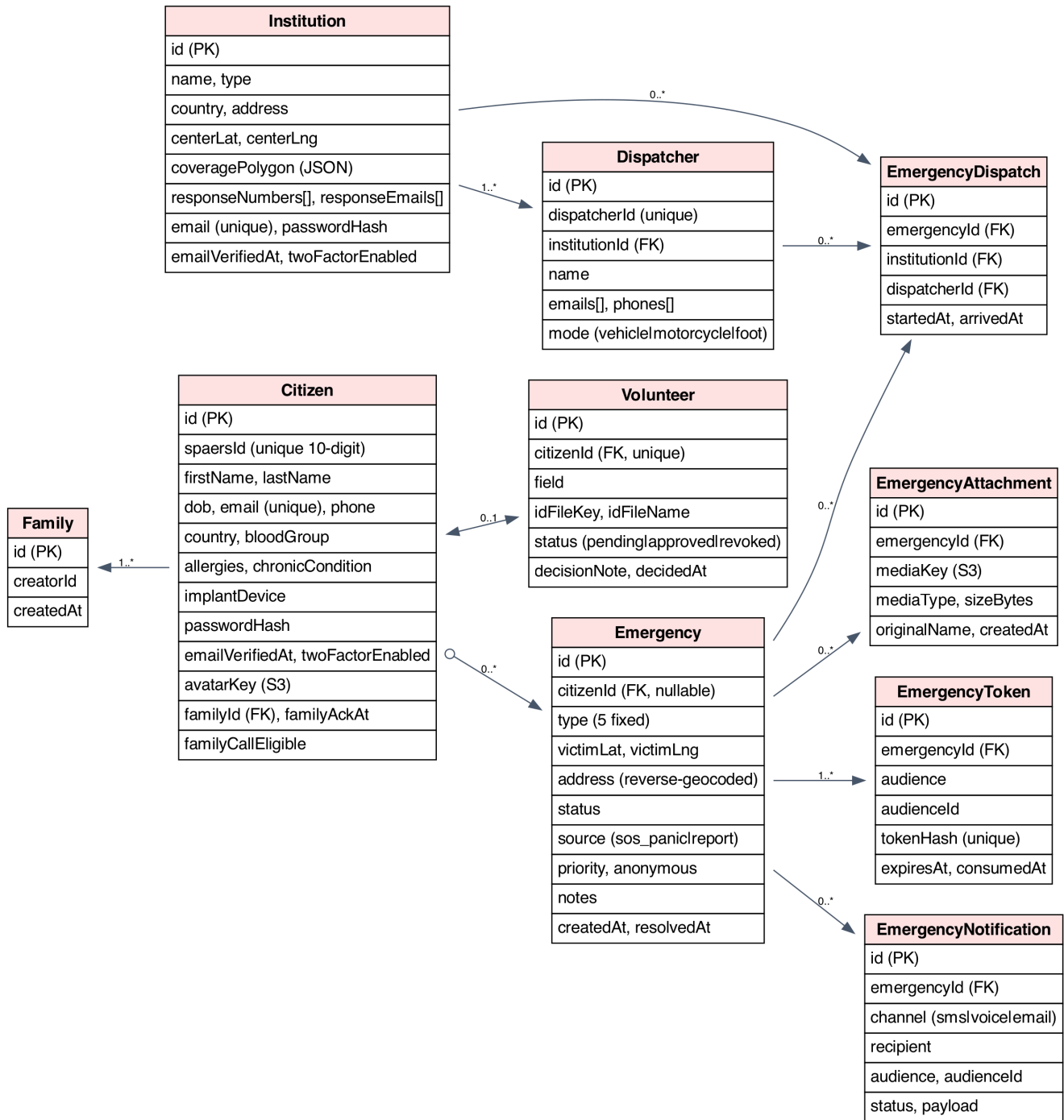


Figure 3.6: Entity-Relationship Diagram (PostgreSQL schema via Prisma)

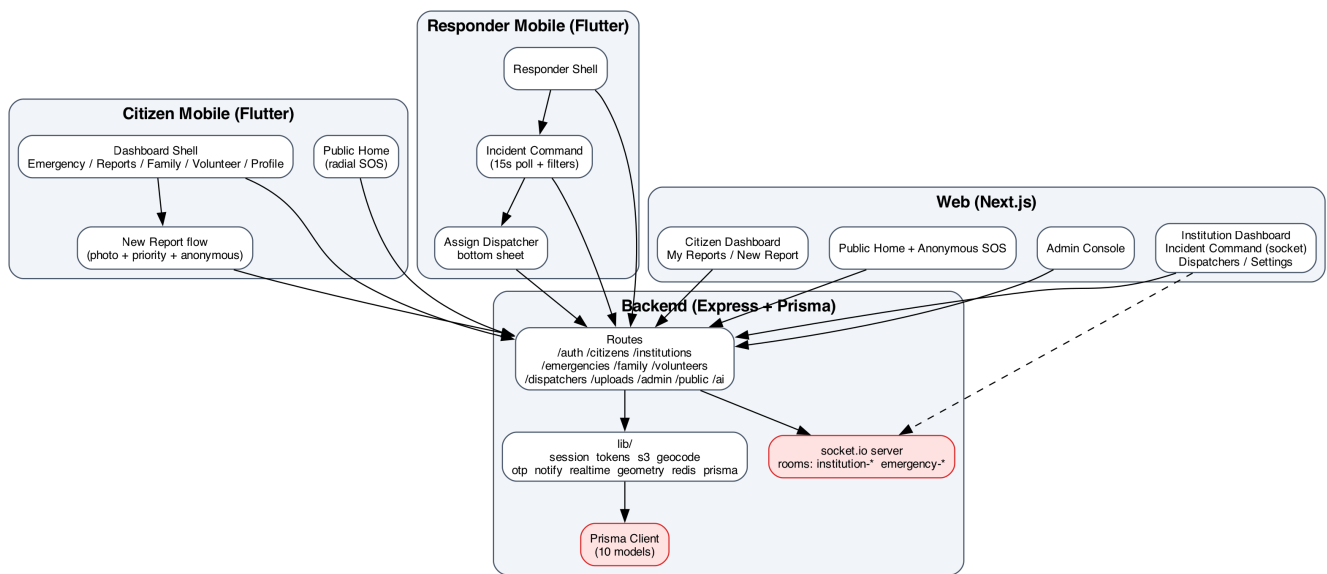


Figure 3.7: Component Diagram

Citizen App	API /emergencies	PostgreSQL	Notifier	socket.io	Institutions
1. press SOS, 5-sec countdown					
2. POST { type, lat, lng } →					
	3. validate session				
	4. INSERT Emergency →	5. row created			
	6. reverseGeocode (fire-and-forget) →				
	7. point-in-polygon scan →	8. matched institutions			
	9. issueToken per institution				
	10. dispatch fan-out →		11. SMS / voice / email		
	12. realtime emit →			13. emergency:created	14. live card
15. 201 + emergency ←					
16. poll status every 4s					

Figure 3.8: Authenticated SOS Sequence

Browser / App	API /emergencies/anonymous	Redis (rate-limit)	PostgreSQL	Notifier
1. POST { type, lat, lng } →				
	2. INCR per-IP minute key →	3. count		
	4. INCR per-IP hour key			
	5. INSERT Emergency (citizenId=NULL) →		6. row created	
	7. mint victim token (4h)			
	8. coverage match (institutions only)			
	9. notify (SMS / email) →			10. queued
11. 201 { token, expiresAt } ←				
12. save in localStorage; render overlay				

Figure 3.9: Anonymous SOS Sequence

Citizen App	AWS S3	API /emergencies/report	PostgreSQL	socket.io
1. pick type, description, priority, photo				
2. POST /uploads/sign →		3. presign 5-min PUT URL		
4. PUT photo →	5. object stored			
6. POST /report { ..., attachmentKeys } →		7. validate desc 10-1000		
		8. INSERT Emergency + Attachment →	9. rows created	
		10. coverage match (institutions only)		
		11. sanitizeAnonymous if anonymous=true		
		12. notify SMS + email (no family/volunteer)		13. emergency:created
14. 201 + emergency ←				

Figure 3.10: Slow-Path Incident Report Sequence

Institution UI	API /emergencies/:id/admin-token	API /public/e/:token/dispatch	PostgreSQL	Notifier	socket.io
1. fetch /dispatchers (once)					
2. tap Assign on Incident Command card					
3. POST admin-token →	4. verify in coverage				
	5. issueToken (30-min, single-use) →		6. tokenHash stored		
7. token ←					
8. POST /public/e/:token/dispatch { dispatcherId } →		9. resolve + consume token			
		10. INSERT Dispatch + UPDATE status='dispatched' →	11. rows updated		
		12. mint dispatcher token, notify dispatcher →		13. SMS / email / voice	
		14. emit emergency:updated →			15. status pill flips In Progress

Figure 3.11: Assign Dispatcher Sequence

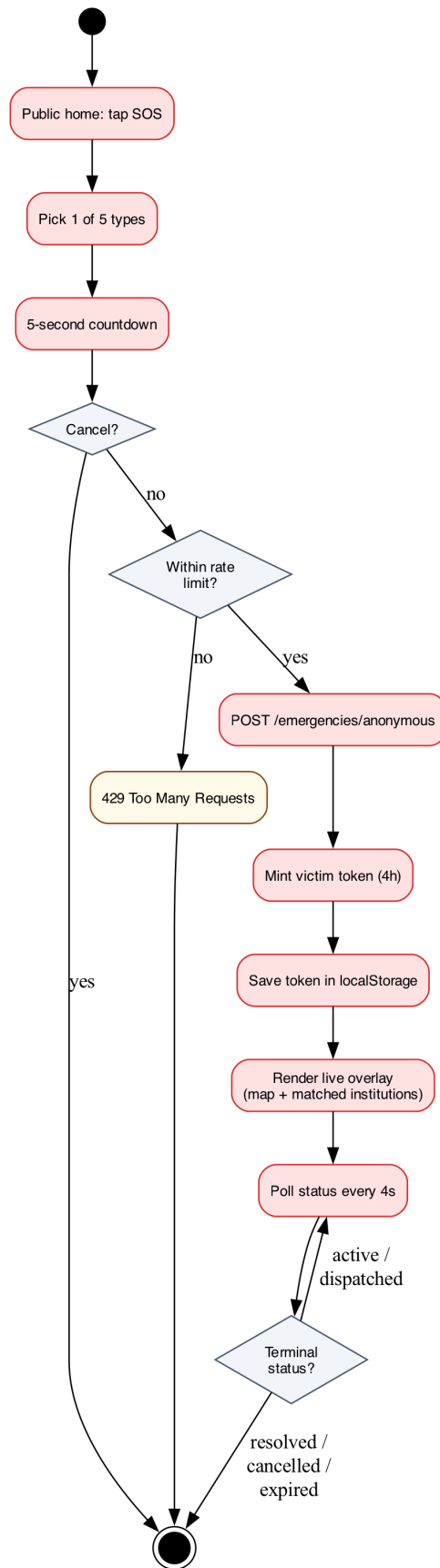


Figure 3.12: Anonymous SOS Activity Diagram